Capstone: the big picture

How a change gets from your VM to production, and which module taught each part. Every arrow is enforced by something, not remembered by someone.

1 Start from your own code
9 findings left in your Terraform
STAGES 1, 2

2 Rebuild it secure by default
private, no SSH, no typed secrets
STAGE 2

3 Automate it so it cannot slip
every scanner a gate, pinned actions
STAGE 3

4 Prove it, change it, tear it down
evidence, a safe failure, a clean account
STAGES 4 TO 6

YOUR VM

You, with Claude Code

Work on a branch, never on main.
Claude Code drafts a change;
you read every line of the diff.

AI · Git

The repo: devsecops-capstone

```
app/                Flask app, tests, Dockerfile
infra/              Terraform, one file per job
.github/            the pipeline
.trivyignore.yaml   the risk register
```

Git · Scripting · Docker · Terraform

Check it on your VM first

pytest and docker compose
semgrep and trivy fs
trivy config and trivy image

Security · Docker

Terraform, run by you

init, plan (read it), apply, destroy
state in S3, locked while you apply
builds everything in the AWS lane below

Terraform · Cloud

git push, open a pull request

GITHUB

Pull request

into main, protected:
a pull request is required
build must pass
no bypass, even for you

Git

build job

every pull request and every push to main

1 secrets: Trivy, first
2 tests: pytest
3 SAST: Semgrep
4 SCA and SBOM: Trivy fs
5 terraform fmt and validate
6 IaC: Trivy config + risk register
7 image: Trivy, CRITICAL with a fix
8 DAST: ZAP, reports only

Every action pinned to a commit ID. Main keeps the scanned image.

CI/CD · Security · Docker

merge

deploy job

main only, never for a pull request

1 waits for your approval in the production environment
2 signs in to AWS with OIDC as the capstone-deploy role: no stored keys
3 pushes the scanned image to ECR, unless it is already there
4 Systems Manager runs the capstone-deploy document on the server:
try it on port 8001, swap onto 8000, or roll back
5 smoke test through CloudFront checks the commit ID

CI/CD · Cloud · Linux

Any gate fails: the pull request cannot merge. Fix it on the branch and push again.

terraform apply builds this lane

AWS

The live environment

Visitor → CloudFront (HTTPS) → Load balancer (80) → App server (8000) → PostgreSQL (5432)

The server and the database are private. No SSH anywhere.
The app reads its database password from Secrets Manager.

Cloud · Linux · Docker · Scripting

pull

ECR

capstone-api images
tagged with commit IDs
that can never move,
scanned on every push

Docker

Systems Manager

Run Command: the one
document the pipeline
may run. Session Manager:
your shell. Parameter
Store: two settings.

Cloud

CloudTrail

every deploy and every
Session Manager login,
with who and when

On by default:
Terraform does not build it.

Cloud · Security

the deploy document runs on the app server, through the agent's own outbound connection

Everything here is code in one repo. terraform destroy removes the AWS lane; terraform apply and Run workflow bring it back in about 20 minutes.